Week 2 Core Functionality

Project: Local Service Problem Diagnosis and Matching System

This report explains the main functions added for the Week 2 milestone. The work focuses on creating a service request and predicting a service category from the problem description. The application was tested in a local browser.

1. Functional Requirements Selected

Two main functional requirements were selected for this milestone. A few related requirements are also supported by the same request workflow.

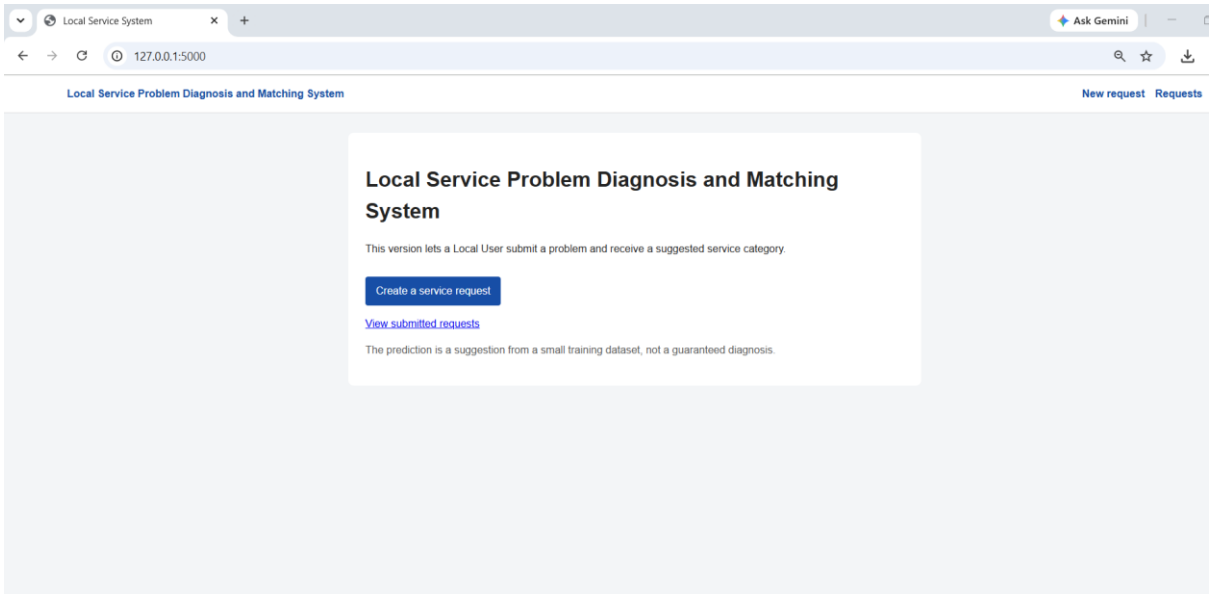
- **FR-04 – Create a new service request.** A Local User can open the new request form, enter a problem description, and submit it. The application checks the description and saves the request in the SQLite database.
- **FR-40 – Predict a suitable service category.** The application sends the problem description to the NLP/ML classifier. The classifier predicts a service category and returns a confidence value.
- **FR-05 – Enter a problem description.** The form allows the Local User to enter details about the household or equipment problem.
- **FR-07 – Submit without selecting a category.** The category field is optional. The user can leave it blank and allow the classifier to suggest a category.
- **FR-10 – Accept or change the suggested category.** The form allows the user to select a category. When a category is selected, the application saves it as the final category while keeping the model prediction separately.
- **FR-42 – Store the predicted and final categories.** The database stores the model-predicted category and the final category in separate fields for later review and model evaluation.

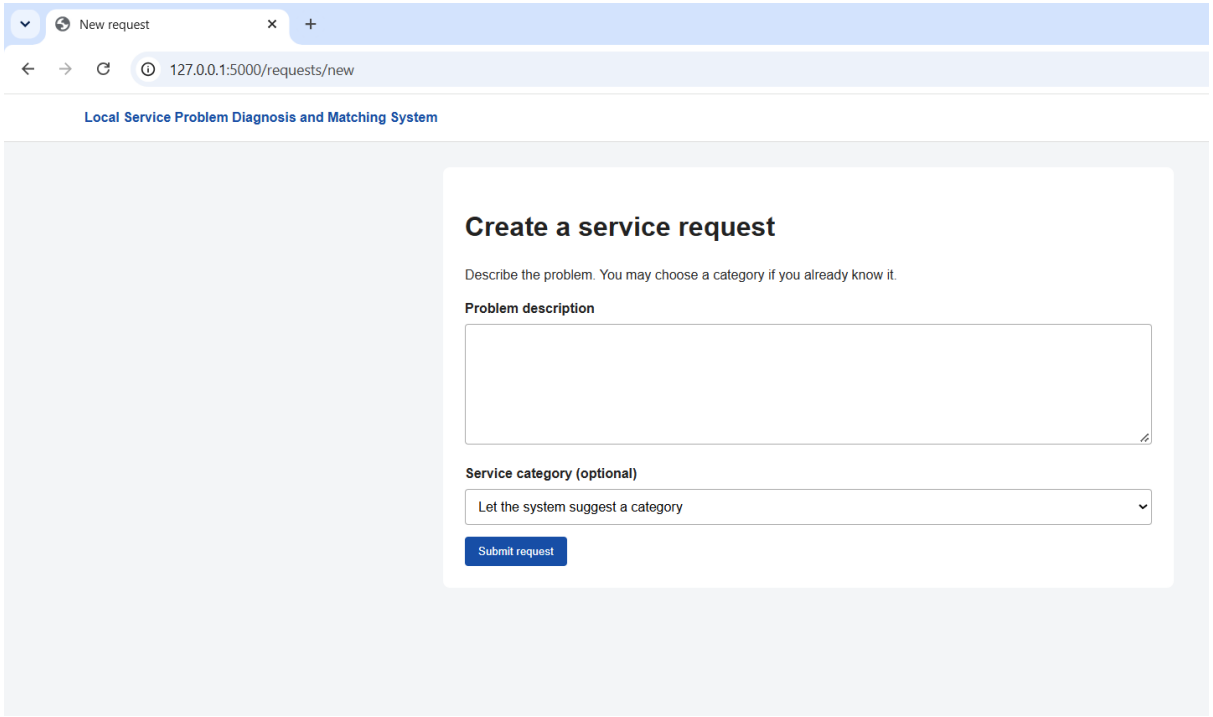
2. Implemented Functionality and Source Components

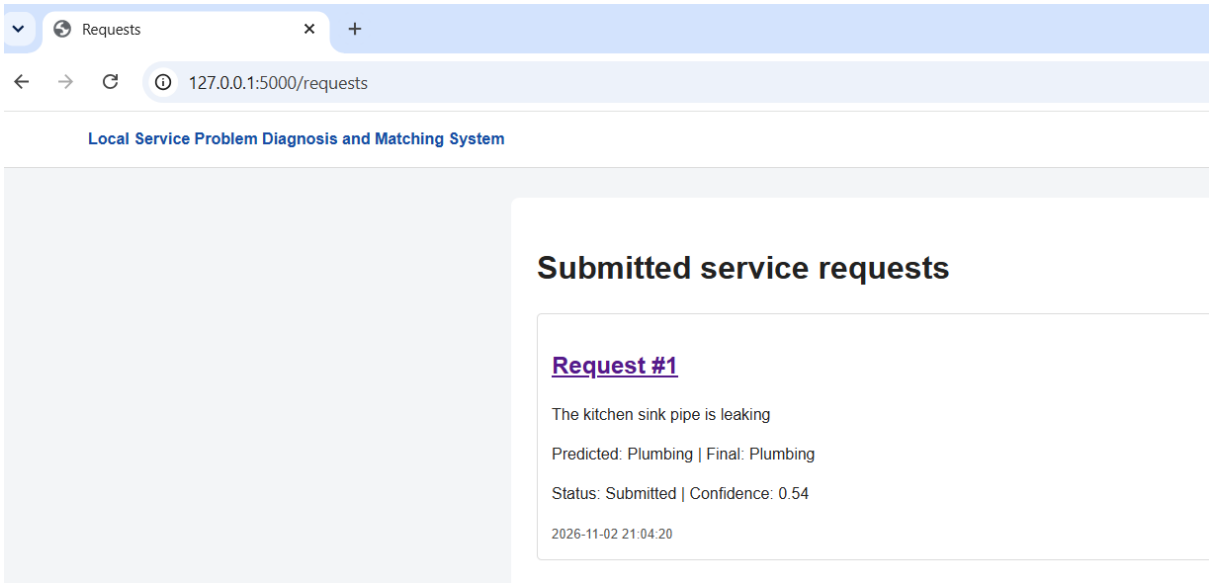
- **app.py —** Runs the Flask application. It displays the pages, receives the submitted form, checks the input, calls the classifier, saves the request, and opens the request details page.
- **classifier.py —** Loads the sample training data and uses TF-IDF with Logistic Regression to predict one of the available service categories.
- **database.py —** Creates the service request table in SQLite, inserts new requests, and reads saved requests.
- **sample_training_data.csv —** Contains sample problem descriptions and their service categories. The classifier uses this data when it is available.
- **templates/ —** Contains the home page, new request form, request details page, and submitted requests page.
- **static/style.css —** Provides the basic layout, colors, spacing, and form styling.
- **requirements.txt —** Lists Flask and scikit-learn, which are needed to run the application.

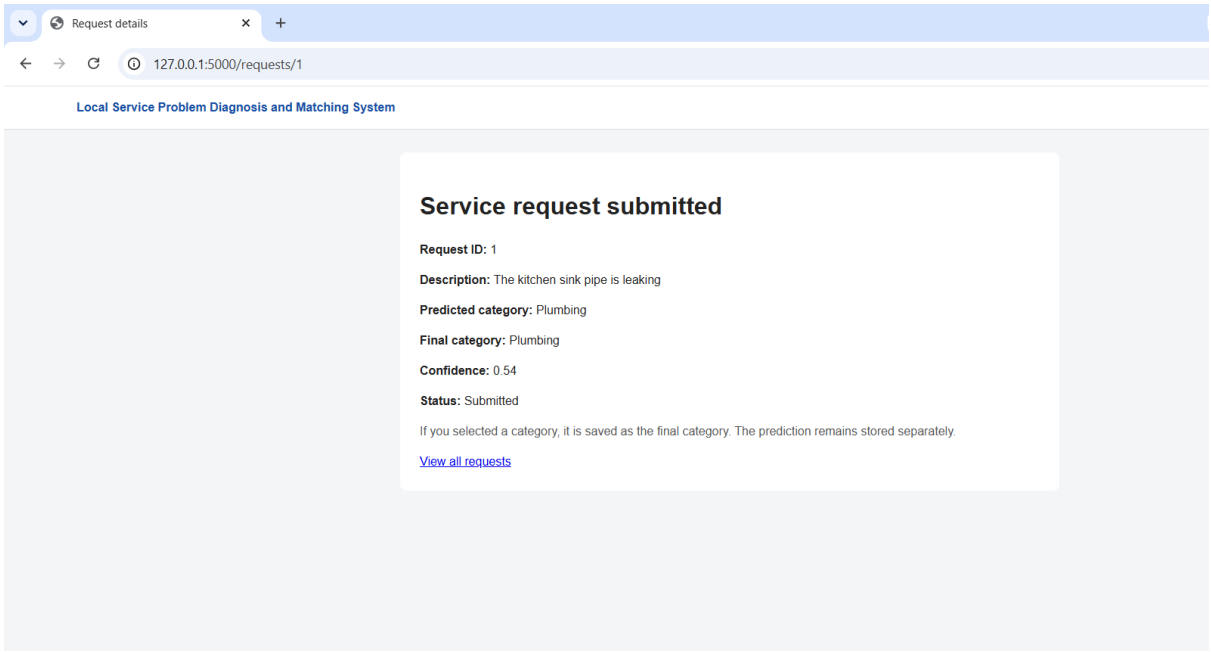
3. Tests Performed and Results

The application was started locally and tested through Google Chrome at http://127.0.0.1:5000. The following results are based on the screenshots shared during testing.









Test	Result	Observed output
Open the application home page	Passed	The home page loaded and showed the New request and Requests links.
Open the new request form	Passed	The form displayed a problem description field and an optional service category list.
Submit a problem without choosing a category	Passed	The description “The kitchen sink pipe is leaking” was accepted.
Predict the service category	Passed	The model predicted Plumbing with a confidence value of 0.54.

Save the service request	Passed	The application displayed Request ID 1 and status Submitted.
View the saved request	Passed	The Requests page displayed Request #1, the description, predicted category, final category, status, and confidence.

The saved request list showed Request #1 with the description "The kitchen sink pipe is leaking," Predicted: Plumbing, Final: Plumbing, Status: Submitted, and Confidence: 0.54. This confirms that the request was available on the list page after submission.

The screenshots confirm the browser tests above. The automated unit tests included in the source package have not yet been reported as run, so this document does not mark those tests as passed.

4. Technical Problems Encountered

During setup, the command to upgrade pip stopped with a KeyboardInterrupt while Python was loading pip's internal files. The issue occurred in the virtual environment and was not caused by the Flask application code. The suggested fix was to repair pip with ensurepip or recreate the virtual environment, then install the project packages.

The project uses a small sample training dataset. Because the dataset is limited, the model may predict the wrong category for some descriptions. The confidence value is a model estimate and should not be treated as measured classification accuracy.

5. Incomplete Work and Further Improvements

- Local User registration, login, logout, and profile management are not implemented.
- Service Provider registration, profiles, categories, and availability are not implemented.
- Provider matching and sending a request to a selected provider are not implemented.
- Provider acceptance, rejection, and request status updates are not implemented.
- Service history, ratings, and written feedback are not implemented.
- Administrator account management, category management, record management, and reports are not implemented.
- The training dataset needs more labeled examples and a separate test dataset.
- Model evaluation using accuracy, precision, recall, and F1-score remains to be completed.
- The automated tests in tests/test_core.py should be run and their actual results recorded.

6. Week 2 Summary

The Week 2 implementation now does more than open a Flask page. A Local User can submit a problem description, receive a suggested service category, and view the saved request. The test screenshots show that the example request was classified as Plumbing and stored in SQLite. The next work will focus on improving the training data, testing the classifier, and adding provider-related functions.